

Introduction to Apache Spark

Big Data Analysis

The Spark Framework

Spark is a platform for cluster computing—meaning that the data and data computations are spread over a cluster with multiple nodes as seen in Figure 1 (a *node* is essentially a separate computer). Spreading the data and workload (computations on the data) makes it easier to work with large datasets because each node only processes a small subset of the data.

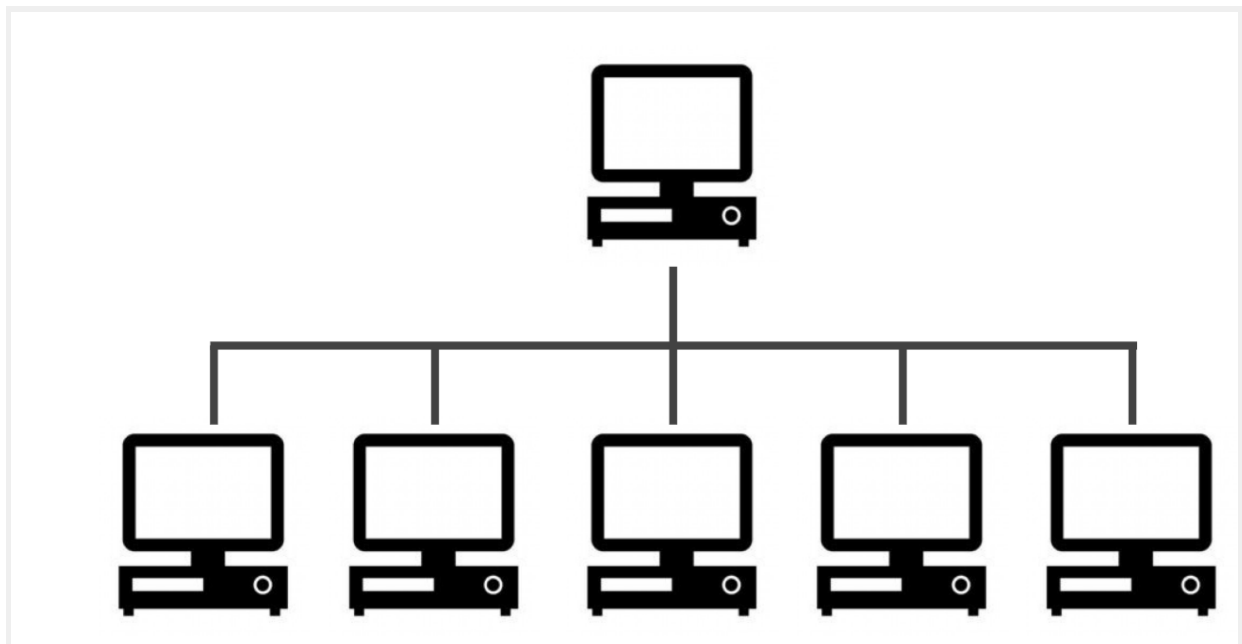


Figure 1: Cluster with multiple nodes

In addition, both data processing and data computation are performed in parallel over the nodes in the cluster. Thus, each node performs only a portion of the overall data processing/computation. Due to this parallelization of the overall workload, the processing and performing of computations on very large datasets are quite performant.

Distributed data processing, and the ability to crunch through large amounts of data in a highly-performant manner, make Spark an ideal platform for big data analytics. Furthermore, Spark provides easy-to-use API's that abstract away a lot of the manual work usually required in big data processing (Pointer, 2020).

Interacting with a Spark Cluster

In a cluster (see Figure 2 below), there is a *master* node that is connected to the rest of the computers in the cluster, which are known as *workers*. The master node orchestrates the processing of the data across the worker nodes in the cluster. The master forwards to the workers the data, and what computations to execute on the data, and the workers send their results back to the master.

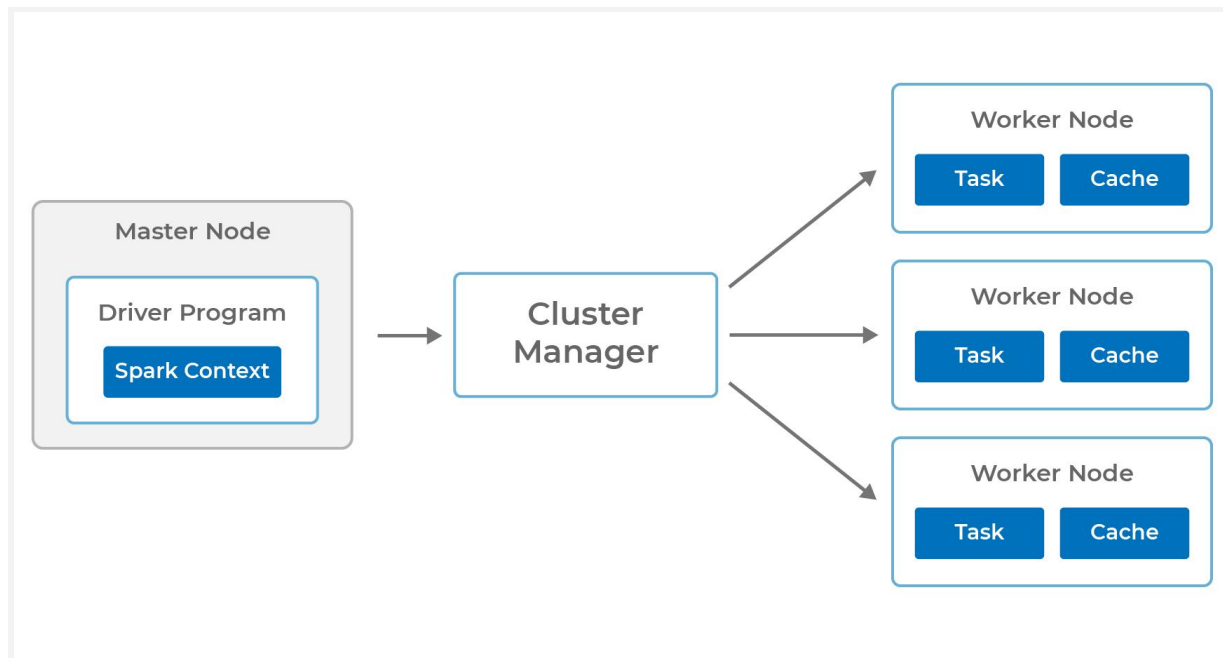


Figure 2: Spark cluster

Within the driver program, a *SparkContext* is created. The *SparkContext* serves as the gateway to all Spark functionalities. Any command you execute against a database goes through the database connection. Similarly, any actions or transformations you perform on a Spark cluster go through *SparkContext*. The *SparkContext* works with the cluster manager to manage various jobs that are spread over the cluster of worker nodes.

Spark Execution Model

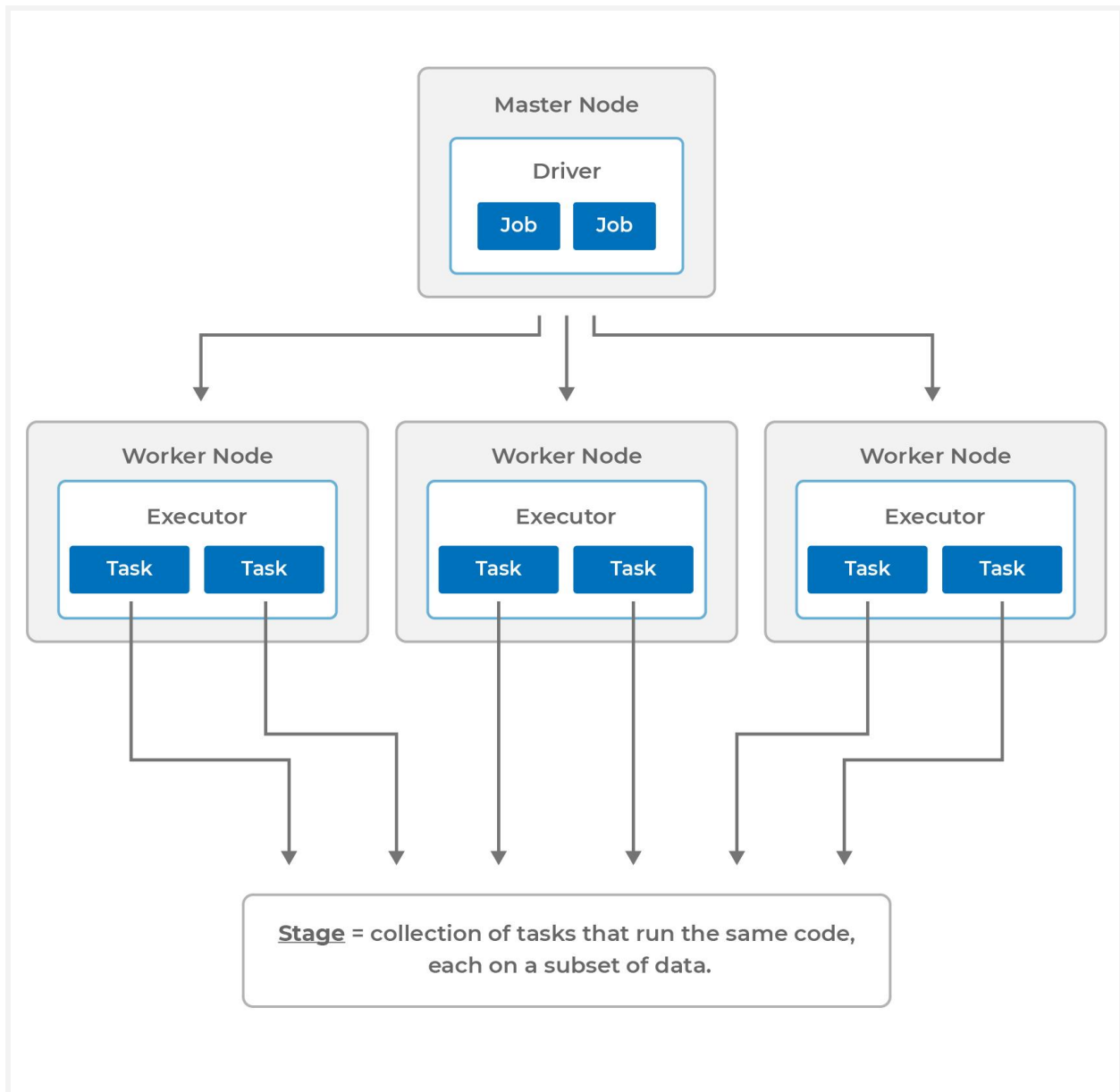


Figure 3: Spark execution model

Below is an example of the process followed in Spark.

Stage 1

When user application code is submitted, the driver transforms the user code into a logically *directed acyclic graph* (DAG)— the transformation of the user code into a DAG occurs behind the scenes and is transparent to the user.

Stage 2

Subsequently, the driver converts the DAG into a physical execution plan that contains a few stages. After conversion into a physical execution plan, the driver program within each stage creates physical execution units called *tasks*. The tasks that constitute a stage are bundled together and then forwarded to the cluster for execution (see Figure 3).

Stage 3

As depicted in Figure 3, the driver interfaces with the cluster manager and requests resources. Once the resources are provisioned, the cluster manager instantiates executors in the worker nodes based on the request of the driver. The driver then forwards the tasks to the executors. As the executors launch, they register themselves with drivers so that the driver has a holistic view of all executors that are involved in the execution of the tasks within a stage.

Stage 4

As the tasks are being executed, the driver program keeps a constant vigil on the set of executors that are turning the tasks. Once the tasks are successfully executed, the results are returned to the drive node via the SparkContext.

Underpinnings of Apache Spark

Apache Spark encompasses *Spark Core* and four libraries: *Spark SQL*, *MLlib*, *GraphX*, and *Spark Streaming*. Individual applications usually require the Spark Core and one of the aforementioned libraries, at a minimum. Previously, separate distributed systems were required to cover the range of workloads that Spark supports. Spark makes processing easy and inexpensive by supporting distinct workloads such as iterative machine learning algorithms, interactive queries, and streaming in the same engine.

Spark SQL

Spark SQL is probably the most programmer-used library/interface when working with and analyzing big data sets, and thus has become increasingly important to the Apache Spark project (Pointer, 2020). As the name indicates, Spark SQL provides a SQL-compliant interface for querying data. In addition, Spark SQL provides standard out-of-the-box support for reading from and writing to other disparate data stores, including Hadoop Distributed File System (HDFS), Apache Hive, JSON, JDBC, Apache ORC, and Apache Parquet. For other popular big data stores, such as CouchDB, MongoDB, HBase, Apache Cassandra, and others, separate connectors are available. (Information on which can be found on the [Spark Packages](#) site.) We will learn more about Spark SQL in the next module.

Spark MLlib

Spark MLlib provides a framework for creating machine learning pipelines and comes pre-packaged with distributed implementations of both supervised and unsupervised machine learning algorithms such as regression, classification, random forest, clustering, etc. Models trained in Python or R can be used seamlessly within a Spark cluster environment and can also be saved for future use in a production environment.

Spark GraphX

[*Spark's GraphX*](#) library gives you the underpinnings to construct a network view that can be leveraged to process large volumes of data. If you were to code something, you would need to construct a graph-like structure from scratch. However, Spark's GraphX library makes it easier to build a network structure with the prepackaged algorithms.

Spark Streaming

Spark Streaming is streaming real-time data, such as IoT data. An example of streaming real-time data is batch data that needs real-time processing. Previously, batch processing and stream processing were deemed as separate processing workloads in Apache Hadoop (Pointer, 2020). Consequently, the MapReduce paradigm was leveraged for batch processing, and an alternate framework/technology (such as Apache Storm) was utilized to support real-time data processing needs. Such a dual processing approach led to disparate codebases residing on completely different frameworks. This required a different set of resources, resulting in operational inefficiencies.

Spark Streaming has converted the concept of batch processing into streaming by decomposing batch data processing into a continuous series or stream of micro-batches where each micro-batch (in turn) can be utilized to facilitate near real-time data processing requirements. As such, Spark Streaming is flexible enough to support both batch and streaming data processing needs on the same framework. This alleviates the operational inefficiencies that resulted from having two disparate frameworks to cater to batch and stream data processing.

Spark's Main Data Structure: Resilient Distributed Dataset

Spark's main data structure is the *Resilient Distributed Dataset* (RDD) which is a low-level object that allows Spark to process large quantities of data by spreading data across multiple nodes in the cluster. RDD's are so integral to the Spark API in that almost every algorithm implemented in Spark involves manipulating RDDs.

The main driver of performance in Spark is the fact that an RDD can be cached in the memory of the compute nodes that constitute the Spark cluster; hence, the RDD can be reused by many iterative tasks. The input data that constitutes an RDD is partitioned into chunks and distributed across the nodes in a Spark cluster. Each node performs computations in parallel on its own chunk.

An RDD can be built on top of data from a multitude of data sources, or it can be constructed from an existing RDD. If for some reason an RDD partition gets lost, it can be recreated using the notion of data lineage; therefore, RDDs are fault-tolerant. Spark tracks the lineage of an RDD by keeping a note of the transformation that has been performed on the RDD. This technique of reconstruction by using the tracked lineage removes the need for costly [data replication](#) strategies used by other frameworks.

A gamut of operations in parallel can be performed upon an RDD. The range of operations falls into two distinct categories: *transformations* and *actions*. Transformations are mainly used to construct new RDDs from sources of data that already exist. Actions trigger computations to compute a resultant value from the data or write the data out to external storage. The transformation operations are 'lazy' in that the computation will not take place when transformations are called. The transformation will only be executed when an action is called to trigger the computation to happen.

Spark DataFrames

The RDD concept has been further augmented with the introduction of *Spark DataFrames*. Spark DataFrames provide a layer of abstraction on top of RDDs. It is effectively a collection of data where data is organized into columns similar to a table in a relational store. In this way, data is represented in a Spark DataFrame in 'column and row' format where each *column* is a feature or variable, and each *row* is a record.

Spark DataFrames can read data from a multitude of data sources. Once the data is within a data frame, several operations (i.e., transformations) can be performed on the data. After one or more transformations are applied, the data is output, displayed, or leveraged for further data processing.

Advantages of Spark over Hadoop

Spark has quickly evolved into the framework of choice when processing big data sets, primarily due to two main reasons:

1. Spark's in-memory data engine performs tasks several times faster than MapReduce, specifically when orchestrating multi-stage jobs that require multiple writes to disk across stages.

2. The second advantage, and probably even more important than improved speed or performance, is the programmer-friendly Spark APIs. The Spark APIs make it relatively straightforward for developers to write sophisticated and complex programs geared towards processing or manipulating big data sets.

References

Bonner, S., Kureshi, I., Brennan, J., & Theodoropoulos, G. (2017). [Chapter 14 - Exploring the Evolution of Big Data Technologies](#). *Software Architecture for Big Data and the Cloud*.

Pointer, I. (2020). [What is Apache Spark?: The Big Data Platform that Crushed Hadoop](#).

Vaidya, N. (2020). [Apache Spark Architecture - Spark Cluster Architecture Explained](#).